What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)
Affiliation
Address
email

Abstract

When a coding agent is graded, only its final patch is examined. We asked what happens in between: does the agent break the repository's existing tests while it works, and does the final patch show it? We built an instrument that snapshots the working tree after every edit and re-runs the repository's previously-passing tests at every snapshot inside the benchmark's own container. In a pilot on 40 SWE-bench Verified tasks, agents broke previously-passing tests 184 times along the way; the final patches showed one of those breaks. Whether this happens depends on the model more than on the task: on the same tasks and harness, one frontier stack never broke a test and another did so 140 times. Rolling back to the last verified checkpoint keeps the final tree clean but costs solved tasks, because the check that triggers the rollback rejects correct patches. Gating each step on the repository's own tests instead removes that cost: 65% solved with no contaminated final tree, and 70% when the gate is graded on tests it never reads. We release the instrument, its validation protocol, and a catalogue of 28 measurement defects met while building it, 23 of which produced plausible numbers rather than errors.

1 Introduction

Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] apply the patch an agent submits, run the repository's tests, and report whether the target tests now pass and the previously passing tests still pass. Work on regressions caused by agents follows the same convention and counts the previously-passing tests the submitted patch breaks [8].

That convention cannot see what happens while the agent works. An agent that breaks an existing test at step three and repairs it at step five submits a clean patch; any harness with a recovery mechanism produces that pattern by design. Two basic questions therefore have no answer: how often do agents break existing behaviour during a run, and how much of it survives to the final patch?

We answer both by measuring the timeline instead of the endpoint. Every mutating tool call becomes a commit on a git reference the agent cannot see; after the run, the repository's previously-passing tests are re-run at every commit inside the benchmark's pinned container. This is the observation primitive an agent operating system needs before it can build a recovery primitive. Our contributions are:

1. **An instrument for event-level regression measurement.** Every mutating tool call is recorded as a commit on a git reference the agent cannot see; after the run, the instance's held-out passing tests are replayed at every recorded state inside the benchmark's pinned container, and detection is attributed by coverage. Validity is established by controls, a

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

liveness gate, a golden check through the agent's own execution path, and a re-scoring control (Section 3).

2. **Measurements on SWE-bench Verified.** Across the GPT-5.6 rollback runs that produced any regression, the timeline recorded 184 events and the final state exposed one (Section 5.2). Event frequency depended on the model stack: identical instances and harness gave 0 events under one frontier stack and 140 under another (Section 5.3).
3. **A pre-registered comparison of recovery policies, and a fix.** With the analysis committed before unblinding, rollback to a verified checkpoint left fewer contaminated final trees than no recovery ($p = 0.022$) but resolved fewer tasks; we trace the loss to verifier precision, and a post hoc regression-gated verifier, enforced by the harness at every step, removes it: 65% resolve with no contaminated tree, 70% in a split variant graded on tests the gate never reads (Sections 5.4 to 5.6).
4. **A catalogue of 28 measurement defects** found while building the instrument, classified by mechanism (Appendix A); 23 produced a plausible number rather than an error.

All measurements are exploratory and were made on a development slice of the benchmark that is excluded from any confirmatory frame. No claim is made about the population of instances beyond that slice.

2 Background and related work

Benchmarks and their validity. SWE-bench [1] grades a patch by tests that must go from failing to passing and tests that must keep passing; Verified [2] is the human-screened 500-instance subset. Concerns about the static subset have accumulated: contamination, memorisation, and leaked solutions [6], flawed tests [7], and rolling [3, 5], private [4], or long-horizon [23] alternatives in response. UTBoost [7] found the held-out tests often insufficient: augmented tests exposed 345 mislabelled patches, affecting 24.4% of Verified leaderboard entries. Wang et al. [37] find resolve rates on Verified overstated by 6.2 points once plausible-but-wrong patches are inspected. TDAD [8] measured regressions in submitted patches on Verified (6.1% of runs under a vanilla open-weight agent) and reduced them with a pre-change impact-analysis map that tells the agent which tests to verify. Process-level benchmarks have begun to score intermediate states: RigorBench [35] scores per-state test stability on 30 curated tasks from logs, SWE-CI [36] tracks regressions across CI iterations of its own tasks, and AgentLens [38] names regression cycles as a lucky-pass mechanism in 10.7% of passing OpenHands runs, from logs alone. We measure the same quantity at every mutating call on standard Verified instances, by executed replay, and attribute each event to the check that could have caught it.

Agent scaffolds and recovery. SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent [12] span the scaffold design space, built on CodeAct [13] and ReAct [14]. Within-episode recovery has been studied as self-reflection [15, 16], progress-gated recovery [17], checkpoint repair [18], provenance-based rollback [19], and aligned context-and-environment checkpoints [41]. Closest to us, Kim et al. [39] re-execute intermediate edits of 16,758 trajectories, find agents that reach a gold-identical patch and then destroy it, and recover those cases with an edit-commit checkpoint; Gao et al. [40] bind verifier evidence to exact code states and preserve verified checkpoints on function-level repairs. Neither counts regressions or compares recovery policies. Running the repository's regression tests during a run is deployed practice: TestPrune [43] hands the agent a minimised suite it may call, for an 8 to 13% relative resolve gain across three scaffolds, and Trae Agent [44] filters candidate patches with regression tests. Operating-system framings [20] move scheduling, context, memory, and storage into a kernel for agents, and a branch-context primitive [42] gives fork, commit, and abort semantics over filesystem and process state; neither observes the tree between actions. Process reward models and rubric verifiers [24, 25] approach the verifier-precision problem of Section 5.5 from the reward and test-time-selection side.

Automated program repair. The regression problem is the plausible-versus-correct patch problem of program repair [26, 28, 29]; regression tests are the main defence against overfitting patches [27]. Our results concern regressions the agent's own process repairs before the patch is tested; trajectory-level evaluation [21, 22] argues that resolve rate alone is uninformative. Public leaderboard trajectories record actions and observations but not the tree at each step, so this measurement cannot be mined from them without re-execution; the instrument commits the tree.

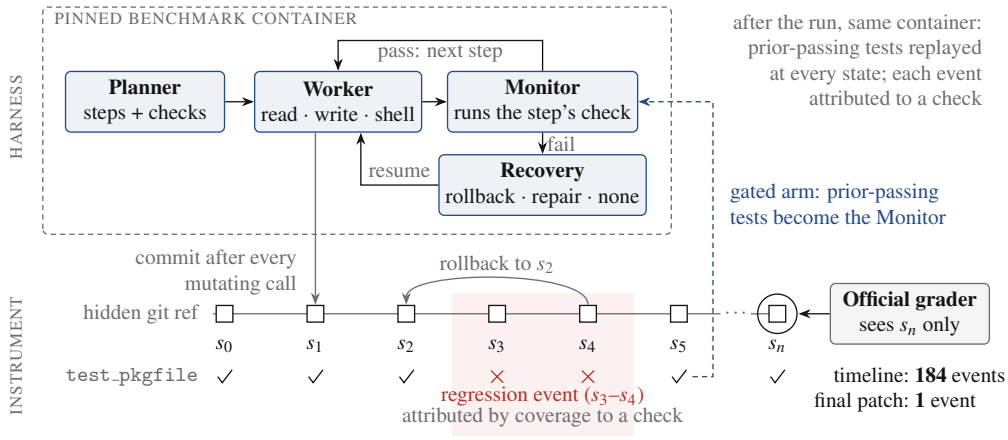

Figure 1: The harness and the instrument. The planner, worker, monitor, and recovery policy run inside the benchmark's pinned container; every mutating tool call commits the tree to a hidden git reference, and after the run the instance's previously-passing tests are replayed at every state. The test row shows one such test regressing at s_3–s_4 and repaired by a rollback to s_2, which the official grader, reading only s_n, never sees. In the gated arm (dashed), the timeline's tests serve as the monitor.

3 Instrument

Figure 1 shows the harness and the instrument. **Observational timeline.** After every mutating tool call, the working tree is committed to a git reference outside the agent's view, using a private index so that the agent's own `git status` and `git diff` are unchanged. Rollbacks and the end of the run are observations too, and because observations are commits, an archived run can be re-measured later without re-running the agent.

Exhaustive replay. For each observation, the instance's passing-test set is run inside the pinned container against that observation's tree; a regression event is a test that passes at one observation and fails at a later one. Every observation is replayed rather than bisected: a recovery policy makes verdicts non-monotone. Replay is scoped to the files holding the held-out tests, which is what keeps it affordable: 4 to 8 seconds per observation on the two repository families we timed (pytest and django), about 26 CPU-minutes for a 40-instance sweep, and a projected 80 CPU-hours for 500 instances at 100 edits each. Infrastructure failures during replay are recorded as missing observations rather than as test failures, and baseline-dead tests are excluded from event counting.

Attribution. A detected regression is classified three ways: *attributed* if a failing harness check and the broken held-out test both exercise a file the agent changed at that observation (using a per-instance coverage map built at the base commit), *co-occurring* if some harness check failed while the regression was open but no such link exists (an over-count of detection), and *unknown* if the coverage map cannot say.

Execution environment. The agent's tools and the harness's checks execute inside the same pinned container as the replay, with file changes synchronised to the host tree the timeline records (Appendix A.2).

Validation. Five gates run before any paid experiment: a negative control, a positive control with injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and a baseline liveness check. A golden check drives the gold patch through the agent's real tool path and requires the grader to return "resolved" (a null run "unresolved"); it passed on three repository families. A re-scoring control re-measures an archived timeline under a changed instrument with no model calls; on both runs to date it reproduced the archived episode counts.

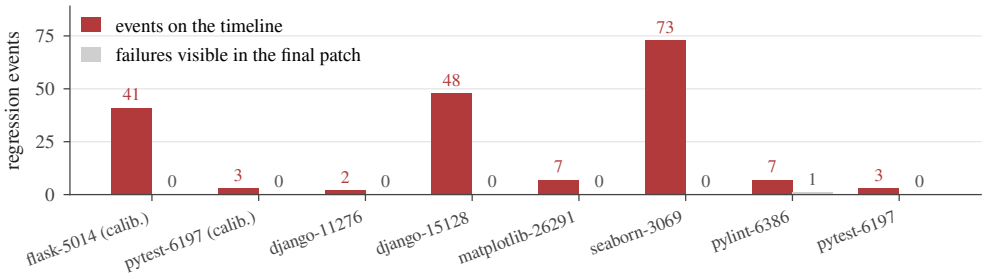

Figure 2: Every run that produced at least one regression event, in both GPT-5.6 sweeps: events recorded on the timeline (red) against held-out test failures visible in the graded final patch (grey).

4 Experimental setup

Benchmark and slice. SWE-bench Verified, 500 instances. It is no longer a capability benchmark: its maintainers' audit found saturation, contamination, and tests that reject correct patches [46]. We use it as a substrate, not a leaderboard: its pinned images and previously-passing test sets make the measurement possible. A 40-instance development slice (16 from django), stratified and fixed before any measurement, was used throughout and is excluded from any confirmatory frame. The GPT-5.6 stack also ran on the slice's first 10 instances as a calibration, and plain rollback ran twice.

Harness. A planner decomposes the task into steps with verification commands; a worker executes each step with three tools (read, write, shell); a monitor runs the verification, and on failure the recovery policy acts: **rollback** (reset to the last verified checkpoint, retry with feedback), **repair-in-place** (retry from the failed tree), or **no recovery** (keep the failed step's tree). Runs have a $4 work-cost cap; exhaustion scores as failure.

Models. Two frontier stacks under identical harness, policy, instances, and caps, named by their API model strings: `claude-opus-4-7` (planner) with `claude-sonnet-4-6` (worker), and `gpt-5.6-sol` (planner) with `gpt-5.6-terra` (worker), recorded in every run manifest.

Outcomes. Resolve is the official grader's verdict on the final patch. Events are reported at three levels because the distribution is overdispersed: distinct incidents (observations at which at least one test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs. Final-state contamination is the number of held-out tests failing in the graded final patch, net of baseline-dead tests.

Pre-declaration. The event unit, the gates, and the contrast were declared before the relevant data existed; the contrast's analysis (exact paired sign tests, final-state contamination primary, incident exposure co-primary) was committed before either comparison arm finished; one amendment, making the gates conditional on the model stack, preceded unblinding (Appendix B).

5 Results

5.1 Resolve and cost

Under rollback, the Claude stack resolved 13 of 40 instances (32.5%, exact 95% CI 18.6% to 49.1%) for a total sweep cost of $34.24; the GPT-5.6 stack resolved 13 of 40 (32.5%; 13 of the 35 graded, 37.1%) for $8.14. Three GPT-5.6 cells were never attempted (circuit breaker) and two were lost to a file-synchronisation defect; all five count as unresolved out of 40 and are excluded from the graded denominator. One instance cannot be resolved by any arm under the current official parser (Appendix A.3), so every rate has a ceiling of 39. Runs are short: the median run makes 5 mutating tool calls (IQR 4 to 8, maximum 19 over 235 runs; 94% make 10 or fewer), an order of magnitude below public-scaffold trajectories.

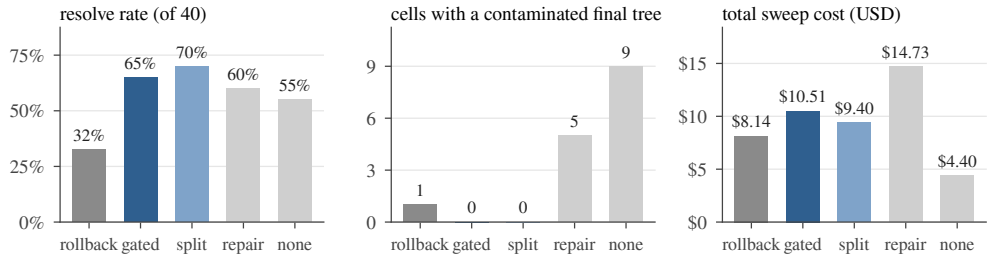

Figure 3: The five recovery arms under the GPT-5.6 stack on the same 40 instances (split: the split-oracle gate). Contamination counts cells whose final patch fails a previously-passing test.

5.2 The final state hides almost all regression activity

Figure 2 shows every run that produced a regression event under the GPT-5.6 stack with the rollback policy, pooling the 10-instance calibration and the 40-instance sweep (47 runs). The timeline recorded 184 declared events across eight runs; the final patch exposed one. Run-weighted, seven of the eight bearing runs ended with a clean final patch; three storms (73, 48, and 41 events) supply 88% of the event count, so the run-weighted figure is the more robust statement.

One resolved run illustrates the mechanism: all 147 graded tests pass in its final patch, while its timeline holds three regressions of the same test function at observations 3, 5, and 7, each repaired by rollback two observations later, two of them undetected by any harness check while open. Across the full GPT-5.6 sweep, 48 of 162 raw episodes had no co-occurring harness failure.

5.3 Regression frequency depends on the model stack, not the benchmark

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2% to 32.0%). A one-sided Fisher exact test on the bearing fractions gives $p = 0.010$; this is a single pairwise test on six bearing runs (reassign three of them and $p = 0.11$), and the two stacks also traverse different provider adapters. Observation density differs only by $1.2\times$. The two stacks resolved the same number of instances (13 of 40 each) for total sweep costs of $34.24 against $8.14: the stack that never broke adjacent behaviour cost four times as much per run. The three storms that supply 88% of the events are model edits (six lines in seaborn's `plot.py`, nine in django's `query.py`, a 262-line deletion in flask's `blueprints.py`), none truncated or from a capped turn (defect D10 was fixed before these sweeps).

The Claude zero is measured, not a detection failure: the same stack's earlier canary run produced three events, and re-scoring its archived timeline under the sweep's instrument reproduces them.

The pre-declared event-rate gate failed for both stacks, so no arm here is a confirmatory pass (Appendix B); we claim no more than a single-sweep dependence on the model stack.

5.4 Recovery policy: rollback keeps the final tree clean

Figure 3 summarises the pre-declared contrast. The primary endpoint, final-state contamination, is paired by instance: no recovery was worse than rollback on 9 instances, better on 1, and tied on 25 (exact sign test, $p = 0.022$). Repair-in-place was worse on 5, better on 1, and tied on 29 ($p = 0.22$). The co-primary endpoint, incident exposure, did not differ detectably (no recovery $p = 0.45$; repair-in-place $p = 1.0$): regressions occurred at similar rates under every policy; the policy determined whether they persisted.

Disclosure. The first unblinding gave $p = 0.375$. Seven cells (five no-recovery, two repair-in-place) whose final trees made the suite uncollectable had been dropped as ungradable, removing the most contaminated states from the endpoint; official grading scores such a patch as failing every test. The grader was corrected to distinguish an environment failure from a patch that kills the suite, and the

seven archived trees were re-graded without re-running any agent, giving p = 0.022. The rule changed after the data were seen (Appendix B).

5.5 Rollback loses resolution to verifier precision

Rollback resolved fewer tasks than either alternative (Figure 3, left): 13 of 40 against 24 (repair-in-place) and 22 (no recovery); paired by instance, it won one discordant pair and lost nine against repair-in-place (McNemar exact p = 0.022) and won three and lost nine against no recovery (p = 0.15). The loss is attributable to the verifier: of the 18 rollback runs that failed, 15 failed at the first step, and 9 of the 18 were resolved under a policy that kept the rejected work: the planner-written check had rejected a patch the grader accepted, and rollback discarded it.

5.6 Regression-gated rollback removes the tradeoff

A fourth arm, added after unblinding and therefore exploratory, replaces the monitor's planner-written check with the repository's previously-passing tests, run in the agent's container at the start and after every attempt; a step is rejected only if a test that passed at the start fails now. It resolved **26 of 40 instances (65.0%)**, the most of any arm and the level frontier scaffolds reach ungated [39, 44], with **no contaminated final tree** (Figure 3). Paired against plain rollback on the 35 shared instances, it resolved 10 that plain rollback did not and lost 1 (McNemar exact p = 0.012; post hoc, unadjusted; onset exposure p = 0.73). The gate's tests come from benchmark metadata: its baseline oracle coincides with the grading oracle (median ratio 1.00), so clean final trees are guaranteed by construction, and the selection itself observes roughly the gold test patch's blast radius, which a deployed agent lacks. The deployed analogue gates on the full suite at baseline (django's takes about two minutes per attempt against 4 to 8 seconds scoped) or on a metadata-free selector such as TDAD's map [8]; the gated rates are upper bounds on either. A fifth arm, a split variant, removes the guarantee: the gate reads a deterministic half of the previously-passing ids (2,278) and never the other half (2,585), on which the grader rules; reading only previously-passing tests also rules out overfitting to a visible target test [45]. It resolved **28 of 40 (70.0%)**, indistinguishable from the full gate (5 discordant pairs against 3, p = 0.73) and 13 pairs against 2 over plain rollback (McNemar exact p = 0.007), with no final tree failing a held-out test, net of the parser-capped instance of Section 5.1. A second plain-rollback sweep reproduced the first (13 and 13 resolved; 6 of 37 bearing both times). Gating on the repository's tests is deployed practice [43, 44]; what is added is harness enforcement at every step rather than a tool the agent may skip, contamination measured beside resolve, grading on tests the gate never reads, and the location of rollback's loss in its verifier.

6 Discussion

Why timeline regressions matter. A regression the agent repairs before submission did not reach the user, but it is not free: 27% of spend across the eight bearing runs ($0.63 of $2.33) went to work done while a regression was open. The events are the per-step ground truth process reward models [24, 25] need; for agent-OS design, a branch context [42] supplies fork, commit, and abort, and the timeline supplies the observation that tells a policy when to abort.

Summary. Final-state evaluation misses the regressions an agent's own process repairs, nearly all of them here; the timeline measurement is feasible, separates recovery policies the final patch cannot, and locates rollback's cost in verifier precision. Building the instrument produced 28 defects (Appendix A); two rules would have prevented most: record infrastructure failures as missing observations, and require a positive liveness signal.

7 Limitations

All measurements come from a 40-instance slice, one sweep per arm, with variability measured only for plain rollback under GPT-5.6. The cross-stack comparison is six bearing runs, confounded with the provider adapter; the gated arms are post hoc and select their tests from benchmark metadata. Runs are an order of magnitude shorter than public-scaffold trajectories (median 5 mutating calls), so recovery dynamics at length are unmeasured, and the ungated 32% baseline is far below frontier

scaffolds'. The held-out tests observe roughly the gold test patch's blast radius, so event counts are lower bounds; a public-scaffold run is the next experiment.

References

[1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.

[2] OpenAI. Introducing SWE-bench Verified. OpenAI blog, August 2024. openai.com/index/introducing-swe-bench-verified.

[3] L. Zhang et al. SWE-bench Goes Live! NeurIPS 2025 Datasets and Benchmarks. arXiv:2505.23419.

[4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.

[5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. NeurIPS 2025. arXiv:2505.20411.

[6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.

[7] B. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.

[8] P. Alonso, S. Yovine, and V. A. Braberman. TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis. arXiv:2603.17973, 2026.

[9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.

[10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.

[11] C. S. Xia et al. Agentless: Demystifying LLM-based software engineering agents. FSE 2025. arXiv:2407.01489.

[12] K. Lieret and C. E. Jimenez. mini-swe-agent. Software, 2025. github.com/SWE-agent/mini-swe-agent.

[13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.

[14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.

[15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.

[16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.

[17] A. Kadu and A. Krishnan. ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.

[18] P. Mazaheri. REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.

[19] Y. Wang et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. arXiv:2606.04990, 2026.

[20] K. Mei et al. AIOS: LLM agent operating system. COLM 2025. arXiv:2403.16971.

[21] S. Kapoor et al. Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation. arXiv:2510.11977, 2025.

[22] R. Shu et al. What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184, 2026.

[23] T. Le et al. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. arXiv:2512.18470, 2025.

[24] M. Raghavendra et al. Agentic rubrics as contextual verifiers for SWE agents. ACL 2026. arXiv:2601.04171.

[25] M. L. Dihan and M. A. R. Khan. SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.

[26] Z. Qi, F. Long, S. Achour, and M. Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. ISSTA 2015. doi:10.1145/2771783.2771791.

[27] Y. Lou et al. When automated program repair meets regression testing: An extensive study on two million patches. ACM TOSEM 33(7), 2024. arXiv:2105.07311.

[28] Z. Fei et al. Patch correctness assessment: A survey. ACM TOSEM 34(2), 2025. doi:10.1145/3702972.

[29] H. Ye, M. Martinez, and M. Monperrus. Automated patch assessment for program repair at scale. Empirical Software Engineering 26(2), 2021. doi:10.1007/s10664-020-09920-w.

[30] SWE-bench issue #601: Test result hijacking via stdout forging in evaluation harness. github.com/SWE-bench/SWE-bench/issues/601, June 2026.

[31] OpenHands issues #4235 (October 2024) and #7044 (March 2025): Docker and environment errors in SWE-bench instance evaluation. github.com/OpenHands/OpenHands.

[32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. NeurIPS 2025 Datasets and Benchmarks. arXiv:2504.21798.

[33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym. ICML 2025. arXiv:2412.21139.

[34] S. Liu et al. Context as a tool: Context management for long-horizon SWE-agents. Findings of ACL 2026. arXiv:2512.22087.

[35] M. B. Madiraju and M. S. P. Madiraju. RigorBench: Benchmarking engineering process discipline in autonomous AI coding agents. arXiv:2606.22678, 2026.

[36] J. Chen et al. SWE-CI: Evaluating agent capabilities in maintaining codebases via continuous integration. arXiv:2603.03823, 2026.

[37] Y. Wang, M. Pradel, and Z. Liu. Are "solved issues" in SWE-bench really solved correctly? An empirical study. ICSE 2026. arXiv:2503.15223.

[38] P. Sahoo et al. AgentLens: Revealing the lucky pass problem in SWE-agent evaluation. arXiv:2605.12925, 2026.

[39] M. Kim et al. Coherence collapse: Diagnosing why code agents fail after reaching the right code. arXiv:2603.24631, 2026.

[40] X. Gao, J. Yang, and Q. Yang. Looping is not reliability: State-bound evidence and typed revision contracts for agentic code repair. arXiv:2607.24604, 2026.

[41] Y. Zhuang et al. AgentRewind: Recoverable execution for long-horizon LLM agents. arXiv:2608.14380, 2026.

[42] C. Wang and Y. Zheng. Fork, explore, commit: OS primitives for agentic exploration. arXiv:2602.08199, 2026.

[43] Y. Chen, T. Ahmed, R. Jabbarvand, and M. Hirzel. Can old tests do new tricks for resolving SWE issues? FSE 2026. arXiv:2510.18270.

[44] P. Gao et al. Trae Agent: An LLM-based agent for software engineering with test-time scaling. arXiv:2507.23370, 2025.

[45] T. Ahmed, J. Ganhotra, A. Shinnar, and M. Hirzel. Investigating test overfitting on SWE-bench. arXiv:2511.16858, 2025.

[46] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. OpenAI blog, February 2026. openai.com/index/why-we-no-longer-evaluate-swe-bench-verified.

A The measurement defect catalogue

Two rules would have prevented most of the defects below: record infrastructure failures as missing observations, and require a positive liveness signal.

A.1 The catalogue by mechanism

Each row is a defect found while building the instrument. **S**: it produced a plausible number rather than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only on a clean host. Class A rows depend on the machine the code runs on; class B rows render an infrastructure failure as a measurement; class C rows measure a state where an event was defined; class D rows consume a producer's output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare `python`	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare `pytest` from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare `python`	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family's oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to 2.7×	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured episodes reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host. Seven further defects found after this census closed are described where they arose (Section 5.4, Appendix A.3, and the repository log).

A.2 The defect that invalidated a conclusion

After nineteen fixes, every validation gate passed, the re-scoring control reproduced archived episode counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared gate.

We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source checkout, while every probe and every gate ran inside the pinned container. On the host, `import matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace package, and everything downstream fails in ways indistinguishable from an incompetent agent. Twenty-six of 28 zero-step runs traced to this. The gates had validated the measurement path and never the agent's execution path. The fix was to route the agent's tools and checks through the same container as the replay, and to add a per-cell environment parity check that runs before any model call.

A.3 Mechanisms in public infrastructure

Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-bench images. Class B: UTBoost's finding that the held-out oracle is insufficient at leaderboard scale [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8]. Class D: the SWE-bench grader accepting forged test output on stdout [30], and, at the harness's current revision, a parser that keys parametrised ids by their full text while the dataset stores them truncated at the first space (the earlier parser's behaviour), so 16 previously-passing ids of one instance in our slice match no output and every submission to it grades as unresolved.

B Pre-declaration and analysis

The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%); the gates were then re-declared per (substrate, model stack) before the contrast was unblinded, and no arm reported here is confirmatory. The contrast's primary endpoint is final-state contamination, paired by instance, exact two-sided sign test; the co-primary is incident exposure. The analysis script was committed before either comparison arm finished. The regression-gated arm was added after the contrast was unblinded and is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored as failure. Instances whose baseline oracle records failures in the unmodified image are excluded from resolve-rate comparability claims, and their dead tests are typed as missing observations for event counting.

Grading correction between unblindings. The first unblinding of the recovery-policy contrast gave $p = 0.375$ for the primary endpoint: seven cells (five no-recovery, two repair-in-place) had been dropped as ungradable because their final trees made the suite uncollectable. Official grading scores such a patch as failing every test, so the most contaminated states had been dropped from the endpoint they bore on most. The grader now distinguishes an environment failure from a patch that kills the suite, and the seven archived trees were re-graded without re-running any agent.